Informed and Assessable Observability Design Decisions in Cloud-native Microservice Applications

Preprint, compiled July 16, 2024

Maria C. Borges[1], **Joshua Bauer**[1], **Sebastian Werner**[1], **Michael Gebauer**[1], **and Stefan Tai**[1]

[1]Information Systems Engineering, Technische Universität Berlin, Germany

Abstract

Observability is important to ensure the reliability of microservice applications. These applications are often prone to failures, since they have many independent services deployed on heterogeneous environments. When employed "correctly", observability can help developers identify and troubleshoot faults quickly. However, instrumenting and configuring the observability of a microservice application is not trivial but tool-dependent and tied to costs. Architects need to understand observability-related trade-offs in order to weigh between different observability design alternatives. Still, these architectural design decisions are not supported by systematic methods and typically just rely on "professional intuition".

In this paper, we argue for a systematic method to arrive at informed and continuously assessable observability design decisions. Specifically, we focus on fault observability of cloud-native microservice applications, and turn this into a testable and quantifiable property. Towards our goal, we first model the scale and scope of observability design decisions across the cloud-native stack. Then, we propose observability metrics which can be determined for any microservice application through so-called observability experiments. We present a proof-of-concept implementation of our experiment tool Oxn. Oxn is able to inject arbitrary faults into an application, similar to Chaos Engineering, but also possesses the unique capability to modify the observability configuration, allowing for the assessment of design decisions that were previously left unexplored. We demonstrate our approach using a popular open source microservice application and show the trade-offs involved in different observability design decisions.

Keywords Observability, Microservices, Software Architecture, Software Design Trade-offs

1 Introduction

The observability practices of monitoring, tracing and logging play an important role in ensuring the reliability of cloud-native microservice architectures. Microservices provide many advantages over their monolithic predecessors [8], however the resulting applications can be difficult to troubleshoot when a fault occurs [20, 29]. This is because each request often travels through multiple independently developed services, deployed on an intricate and evolving stack of software platforms. To address this challenge, observability systems have evolved accordingly, now consisting of distributed services that collect observability data from all levels of runtime platforms as well as from the application level through built-in and developer-defined instrumentation. Hence, using these observability systems requires a multitude of hard decisions in terms of instrumentation points, observability services and configuration. When employed correctly, observability can help developers identify faults quickly, but improper settings can also obfuscate faults [5, 29] and increase latencies and cost.

Still, designing the observability of microservice applications is not trivial. It implies often overlooked tasks like the tool-dependent configuration of parameters, setting alerts and adding custom instrumentation code. These decisions need to be weighed against observability-related trade-offs, e.g., the performance overhead on the application, plus the cost overhead associated with operating the observability infrastructure. However, these architectural design decisions are not supported by systematic methods. Instead, decisions are made based on previous experience and "professional intuition" of the practitioners [28, 29], or developers may end up eagerly instrumenting and changing configuration parameters in a reactive manner after a problem occurs [20]. To weigh between different design alternatives and arrive at an appropriate configuration that justifies the effort, practitioners must have a method to assess the effectiveness of their observability.

Consequently, the need to evaluate the degree of observability of a system has been recognized by research and industry alike [1, 9, 27, 28]. With suitable metrics, it would be possible to obtain a concrete understanding of the quality of the observability, which could then be tracked and continuously improved over time. However, observability has been challenging to quantify so far [27]. Most research has focused either on qualitative assessments of observability tooling [12], or solely looks at cost and overhead [6, 7, 22], failing to address the question of *effectiveness* of observability. Currently, there are no mechanisms for practitioners to quantify or compare the observability of their applications.

In this paper, we argue for a systematic, reproducible and comparative assessment of observability design decisions. Our focus lies on the observability of faults specifically. Similar to software quality measures like test coverage, we aim to make fault observability a testable and quantifiable system property. This, in turn, would help developers identify unobservable faults in their application, thus guiding the configuration and instrumentation process.

Towards this goal, we present the following contributions:

1. A model for understanding the scale and scope of observability design decisions.
2. The concept for testable and quantifiable fault observability metrics.
3. Design, implementation and demonstration of OXN, a tool to automate the process of observability assessments.

The paper is structured as follows: Section 2 discusses related work. Section 3 provides background and models the observability design space of cloud-native architectures. Section 4 presents our observability metrics and experiment method. Section 5.1 introduces our supporting experiment tool OXN. Section 6.3 evaluates our method and tool through exemplary experiments. Section 7 discusses suitability and limitations. Section 8 concludes with future work.

2 RELATED WORK

When faults occur in complex microservice compositions, they can be more complicated to identify than in traditional monoliths, so observability practices have evolved accordingly (e.g., [13, 17, 24]) and observability remains a growing subject in reliability literature. Multiple design decisions have to be made when implementing and using observability tooling. Different model-driven approaches have been proposed to automate the implementation of observability designs [14, 21], yet none of these approaches help practitioners arrive at appropriate and assessable decisions.

Observability design decisions have significant consequences because, as Niedermeier et al. [20] point out, "careless deployment and configuration of monitoring agents have been mentioned as potential problems which might cause instability and an increasing network load". So far, different approaches have been proposed to deal with this design challenge.

Haselböck and Weinreich [10] propose decision guidance models for microservice observability decisions. These models help developers with the more abstract high-level decisions, e.g., what tool to adopt when the goal is to inspect service interactions. The model serves well as an introductory resource, but it can't provide assistance with more intricate configuration or instrumentation decisions. For instance, it does not offer guidance on selecting appropriate metrics or determining the ideal sampling rate. In a similar vein, [23] provide best practices for supporting tracing design decisions. Besides these guides, developers can also refer to surveys like [12] for making decisions, where observability tools are compared based on a number of different qualitative criteria.

Some research also addresses observability design trade-offs, in particular the costs or performance overhead incurred. Ernst and Tai [7] propose an offline approach to tracing overhead assessment. The model-based approach generates realistic trace data, which can then be used as a workload against different tracing backends. A concrete benchmark of observability instrumentation has also been conducted by [22]. Here, the researchers propose a tool for continuous measurement of the overhead of popular instrumentation libraries like OPENTELEMETRY and KIEKER. More recently, the energy efficiency of observability tools has also been investigated [6]. The researchers discovered a close association between observability-related energy consumption and performance overhead, an outcome that was anticipated. All three of these papers address the drawbacks of observability but overlook the advantages it offers. It is not feasible to evaluate trade-offs without understanding the value provided by something.

Our work is the first to guide observability design choices through comparative measurement of its effectiveness. Ahmed et al. [1] did similar work, as they measure how effective four different monitoring tools are at identifying performance regressions. However, their work primarily focuses on comparing the tools without delving into the details of instrumentation and configuration decisions necessary for implementing each tool, as only the default configuration and out-of-the-box automatic instrumentation is used for the assessment. Another promising approach to improving resilience and testing the configuration of the observability systems is Chaos Engineering. It provides a method [2] and tools [11, 18, 26, 30, 30] for carrying out resilience experiments. Here, faults are injected into a running microservice system to test a hypothesis on the systems ability to withstand these faults. However, this approach presupposes a sufficient level of observability prior to conducting the experiments and does not offer any guidance on how to achieve this.

Lastly, Nedelkoski et al. [19] highlight the lack of datasets incorporating telemetry data from diverse sources (metrics, traces, logs). They propose a solution by developing a microservice system that simulates injected faults, enabling the creation of new datasets, similar to our approach. Unfortunately, the system's inability to change or configure observability and specific telemetry instrumentation points greatly limits its ability to assess observability effectiveness.

3 MODELING OBSERVABILITY DESIGN DECISIONS

Ensuring the reliable operation of microservice-based applications is a complicated task that requires observability. Observability, as defined in literature, is the ability to measure the internal state of a system by its outputs [20]. The process of defining these outputs is what we define as "observability design decisions". In this section, we delve into the intricacies and nuances of these decisions by modeling a typical cloud-native microservice application deployed in accordance with modern practices. With this model, we aim to show the scale and scope of observability design decisions, plus it will serve as a common language for practitioners to understand and discuss observability design alternatives. Figure 1 shows our model, which we break down in the following paragraphs.

A **Cloud-native Application** is deployed in a `Cloud Environment` managed by a third-party provider. The model represents the application topology as one or more deployment clusters (`Cluster`), deployed in a specific region and managed by an orchestrator. The orchestrator is a deployment technology, e.g. Kubernetes[1] or Docker Compose[2], that automates the deployment and management of `Containers` across virtual machines (`VMs`). Each container runs an instance of a `Microservice`. We model a microservice as a composition of

[1] `https://kubernetes.io`
[2] `https://docs.docker.com/compose/`

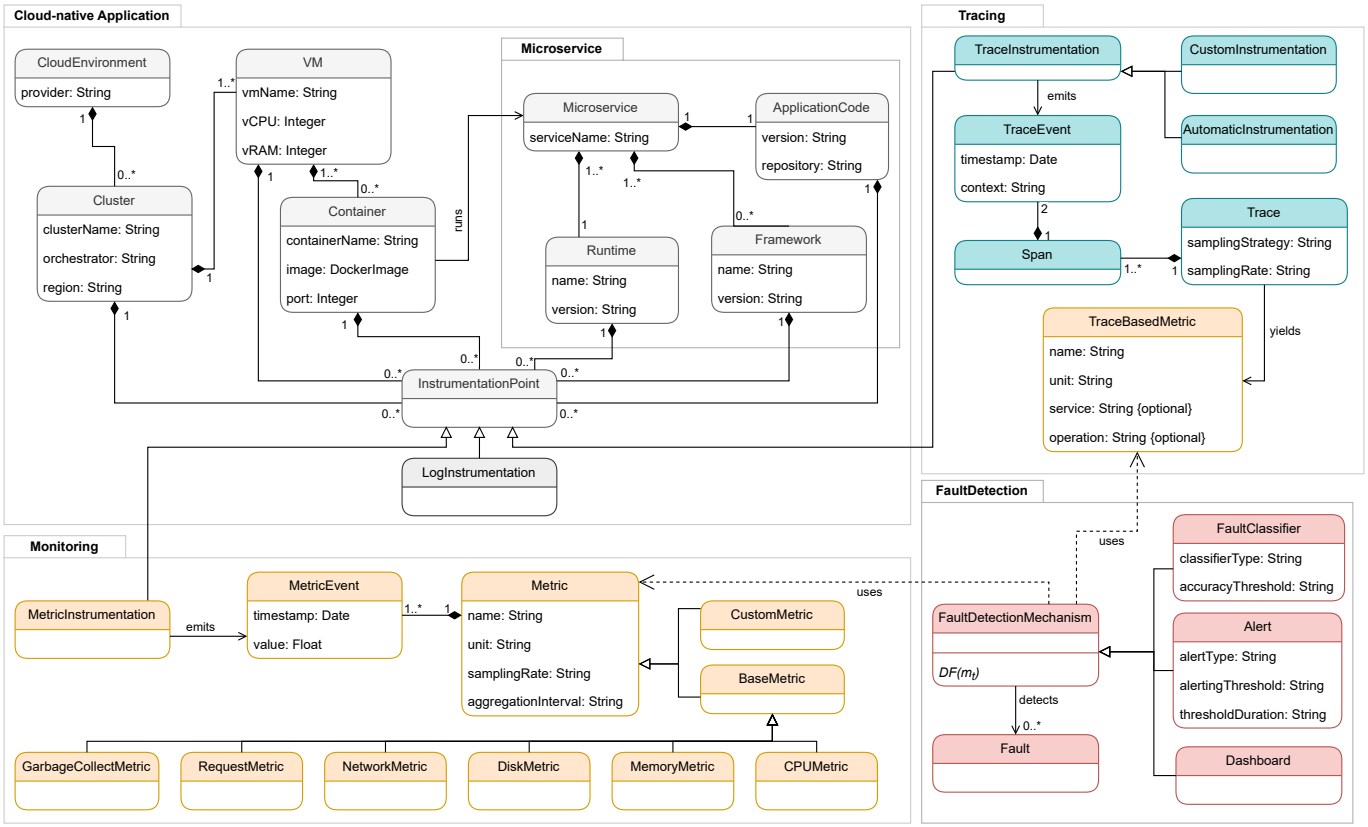

Figure 1: Model of Observability Design Decisions in Cloud-Native Applications

a `Runtime`, which provides the language-specific environment of the microservice, e.g. the Java Runtime. Microservices can also be composed of a number of `Frameworks`, which offer reusable components that streamline the development process, e.g., Django or gRPC. Lastly, microservices are also composed of the `Application Code` written by the development team.

In order to capture observability data, a cloud-native application needs to be instrumented through so-called `InstrumentationPoints`. Instrumentation refers to the process of embedding code within a software component that enables it to collect and emit metrics, traces and logs. Some components of the cloud-native deployment stack are equipped with so-called automatic instrumentation, meaning that instrumentation points are provided out-of-the-box. Depending on the observability data they collect, `InstrumentationPoints` can be of type `MetricInstrumentation`, `LogInstrumentation` and `TraceInstrumentation`. Each instrumentation type supports a different observability practice. While logging is similarly important for building reliable software systems, we mainly focus on modeling monitoring and tracing in this, as logging is less standardized to make a generic model beyond timestamped, human-readable text messages.

Monitoring is the continuous measurement of quantitative properties of a system over a period of time. `MetricInstrumentation`-Points emit `MetricEvents` which are timestamped value measurements. A `Metric`, in turn, represents the time-series collection of several `MetricEvents`. They may be simple `BaseMetrics`, collected directly through automatic instrumentation points provided in `Clusters`, `VMs`, `Containers`, `Runtimes` and `Frameworks`. In Figure 1 we show several of these types of `BaseMetrics`, but the list is not intended to be exhaustive. `Metrics` may also be specified and instrumented by developers in the case of `CustomMetrics`. While designing the monitoring of an application, practitioners face several design decisions, including (1) what `BaseMetrics` to collect from the available automatic `InstrumentationPoints`, (2) whether to add `CustomMetrics` and if yes, where to add the `InstrumentationPoints`, (3) how to configure the attributes `samplingRate` and `aggregationInterval` for each `Metric` instance.

Tracing enables end-to-end observation of application behaviour by retrieving and aggregating event data in so-called traces. In this context, a `TracingInstrumentation`-Point, which can be of type `AutomaticInstrumentation` if supported natively by the microservice `Runtime` or `Framework`, or of type `CustomInstrumentation` if added to the `ApplicationCode` by the development team, emits a `TraceEvent`. A pair of `TraceEvents` demarcating entry and exit build a `Span`, and multiple spans form a `Trace`. The information inside traces can further be processed into `TraceBasedMetrics`, e.g. the duration of spans or of entire traces. While designing the tracing of an application, practitioners again face several design decisions, namely (1) where to add `CustomInstrumentation` and (2) how to configure the attributes `samplingStrategy` and `samplingRate`.

For **Fault Detection**, observability systems employ so-called `FaultDetectionMechanisms` to be able to detect `Faults`. These mechanisms can employ a variety of detection methods, including pre-trained `Classifiers`, `Alerts`, or by having an trained site-reliability engineers look at `Dashboards`. For fault detection, practitioners first (1) need to select a feasible `FaultDetectionMechanism`, then (2) set appropriate attributes to not overload reliability teams and yet provide fast response to potential failures. Naturally, the prior design decisions on logging, monitoring and tracing strongly influence the quality of these detection methods, as too few inputs may lead to a low sensitivity, and too many observations lead to too much noise and thus may lead to oversensitivity.

4 APPROACH TO QUANTIFY OBSERVABILITY EFFECTIVENESS

In order to arrive at informed and assessable observability design decisions, we need to be able to quantify their effectiveness. Observability serves as a necessary precursor for measuring many other system qualities, such as performance [1] or cost [15], but observability itself is very challenging to quantify [27]. Its effectiveness is intricately connected with the very aspects it aims to capture. In this work, we look at observability for the specific purpose of reliability assurance, i.e., the process of ensuring that the system is running despite failures occurring.

Reliability assurance centers around two key metrics: (1) mean-time-to-failure / mean-time-between-failures ($MTTF/MTBF$) and (2) mean-time-to-restore ($MTTR$). $MTTR$ can be further divided into (2.1) mean-time-to-detect ($MTTD$) and (2.2) mean-time-to-repair ($MTTRepair$). The goal of maintaining a high $MTTF$ mostly falls beyond the domain of observability and instead relies on thorough testing. $MTTR$, conversely, is influenced by the presence of observability measures.

Still, $MTTD$ and $MTTR$-like metrics aren't specific to observability mechanisms but apply to the entire operation process, including any other fault-tolerance mechanisms implemented. They serve to track the long-term effect of reliability measures and of reliability teams but are too coarse and can be influenced by many different overlapping factors. For example, they do not indicate whether a failure could have been detected earlier or at less cost with a different observability configuration. To arrive at a systematic method for observability design decisions, we need to measure the impact of configuration and instrumentation decisions. Hence, we need to broaden our perspective beyond time-based process metrics. In the following, we propose a first set of fine-grained metrics as a basis of such a systematic method.

4.1 Concept: Fault Observability Metrics

To arrive at testable and explicit metrics for observability design decisions, we restrict our scope to the visibility of faults, or what we call **fault visibility** [5]. Fault visibility can be defined as the degree to which the data produced during the occurrence of a specific fault is distinct enough from normal operation to trigger a fault detection mechanism. Intuitively, faults that produce very distinct data are easier and faster to troubleshoot. For **fault visibility**, we consider a set of faults $F = \{f_1, f_2, ..., f_l\}$, a set of observability metrics or traces $M = \{m_1, m_2, ..., m_n\}$ and a detection mechanism d. A fault f can be considered visible in metric m if the data recorded during the occurrence of the fault, represented by the time interval $[t_0 - t_1]$, is significantly different from the data recorded under normal conditions, in time interval $[t_{-1} - t_0[$, and thus detected by detection mechanism d. Refer to Figure 2(A) for a visual representation of these time intervals. As modeled in section 3, the detection mechanism can be of different types, e.g., a pre-trained classifier. We consider fault f visible in metric m if detection method d is able to meet its detection threshold and detect the fault. If the detection mechanism is not able to detect the fault, it suggests that either (i) metric m is unsuitable to detect f (ii) the parameters of metric m are not set appropriately or (iii) the function parameters of the detection mechanism d are not tuned correctly. Fault visibility for fault f in metric m through detection mechanism d can thus be defined as

$$v_{f,m,d} = \begin{cases} 1 & DF(m_t) > \alpha \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

where DF represents the detection function of d, α the configured threshold and m_t a subset of observations of m for time t.

The visibility score can be determined for each fault and metric pair, allowing us to assess the impact of individual faults on specific metrics given their current configuration. However, these individual scores are not able to provide a comprehensive and generalized understanding of the observability of the system as a whole. To accomplish this, we can construct aggregate or composite scores. We propose two composite scores: fault coverage and overall fault observability. **Fault coverage** shows the degree to which a fault f is visible across the set of different collected metrics M. We define it as the ratio between the number of collected metrics and a number of metrics where fault f is visible. If a fault is not visible in any metric of the system, it implies that this fault is not covered by the system's observability, and thus the fault coverage for fault f is 0.

$$FC_{f,d} = \frac{1}{n} \sum_{i=1}^{n} v_{f,m_i,d} \tag{2}$$

Lastly, the **overall fault observability** shows the ratio between the theoretically observable faults versus the faults that were actually observed or detected by detection mechanism d. This ratio can improve over time, as developers add more metrics to the instrumentation, or tune the configuration. It can then be defined as

$$OFO_d = \frac{1}{l} \sum_{i=1}^{l} 1_{\{FC_i > 0\}} \tag{3}$$

Figure 2(B) illustrates our suggested fault observability metrics using an example. The main goal for developers should be to increase the OFO. To achieve this, developers have three options, either increase the visibility of invisible faults by changing the configuration of existing metrics, add additional metrics that are more sensitive to the invisible faults, or make changes to the detection mechanism. To weigh between different options, developers need to consider the trade-off between observability overhead and cost. Thus, as a last metric for observability effectiveness, we propose a cost metric. Here, related

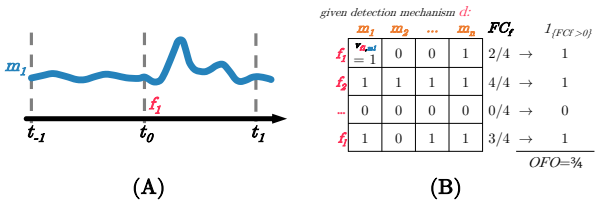

Figure 2: (A) Visualization of the fault model, used to define the metrics in (B)

work [6, 7, 22] has already provided different methods and approaches to quantify the overhead of observability systems. For the purposes of this paper, we will use CPU utilization as a measure for cost, but other metrics such as memory and storage overhead are also suitable.

4.2 *Approach: Observability Experiments*

We propose an empirical and systematic approach to navigate the observability design space: observability experiments. This is a formalized process of experimenting inspired by Chaos Engineering. In Chaos Engineering, faults are injected into a running microservice system to test the system's ability to withstand faults. In our approach, rather than focusing on whether the system survives a fault, we are particularly interested in analyzing the quality of observability data generated during such fault experiments. This quality can be quantified with the metrics proposed in the previous section, and can be measured across various observability configurations and instrumentation alternatives. Practitioners can use observability experiments as a feedback loop to assess observability design decisions, specifically evaluating which changes in the instrumentation machinery and configuration improve or degrade the metrics, all the while keeping an eye on the cost of these changes.

In the following, we describe the approach briefly. An observability experiment formally consists of a description of the system under experiment (SUE), a workload, a set of treatments and a nonempty set of response variables.

System Under Experiment (SUE) In the context of observability experiments, the SUE is either the entirety of a system or a subset of it, i.e., only a select number of microservices. It is often unnecessary to deploy the entire microservice architecture when developers are only interested in studying the observability and instrumentation of a single service and its dependent services. Therefore, it is important to enable experimentation on subsets, especially considering that modern microservice architectures can quickly grow to hundreds of services [16].

Workload We require load generation to simulate users that create realistic observability data. Because the load can affect the data points of the observability data, we include it as a component of the experiment. Further, as a lot of the metrics devised in the previous section rely on comparative analysis, it is important to also have a way to generate a comparable load repeatedly.

Treatments are controlled changes to the system under experiment. Our conceptualization is broader than in Chaos Engineering, where treatment is chaotic or destructive. This need not be the case with treatments in observability experiments. We distinguish between fault treatments and instrumentation treatments. Instrumentation treatments allow users to easily change observability configuration without having to tinker with the SUE code and are executed during compilation. Fault treatments, in turn, are applied at runtime and change the SUE by means of dynamic fault injection. Treatments allow us to investigate changes to the system typically beyond the scope of Chaos Engineering. For instance, the experiment operator might be interested in investigating if an increase in the sampling interval at some metric instrumentation point results in an increase in accuracy for a fault detection model.

Response Variables in observability experiments are metrics or distributed traces and represent the different types of observability data that a system can emit. They are used to calculate the fault visibility, fault coverage and overall fault observability. Responses are decoupled from treatments. This means that we can define a response variable that is not directly related to a treatment, i.e., we could observe a response at service A while treating service B. The decoupling allows us to take into consideration higher-order effects of treatments. For example, we might wish to investigate whether injecting a network delay into service A consequently also affects the throughput at other services that depend on A.

Such experiments are especially feasible in staging environments for systems where both the SUE and the observability system are described using infrastructure as code. In the following, we present a system to automate the process of observability assessments for microservice-based applications.

5 OXN: OBSERVABILITY EXPERIMENT ENGINE

In this section, we present OXN, an extensible software framework to run observability experiments[3]. OXN follows the design principles for cloud benchmarking [3, 4, 25] and thus particularly strives for portable, repeatable, and relevant experiments. We built OXN around a YAML-based configuration file that allows experiments to be shared, versioned and repeated. The experiment configuration describes the SUE, i.e., deployment, treatments, workload and response variables to collect (see section 4.2). Using this single experiment file OXN orchestrates the complete experiment (see Figure 3), by first building and deploying the SUE as well as the load-generator. We use an extendable *runner* component to execute different treatments, such as killing a service container, in combination with the automatic collection of experiment response variables through an observer component. For that, we partly rely on the SUE to implement the OPENTELEMETRY standard, with tools from the CNCF stack[4], namely JAEGER and PROMETHEUS.

5.1 *Architecture*

The design of OXN is modular, decoupled, and extensible. At its core, OXN combines an *orchestrator* to manage the SUE, a *runner* to enact treatments, a *load generator* to stress the SUE and *observers* to capture results. These components all use existing, well-tested software, which we loosely coupled together to

[3] https://github.com/nymphbox/oxn
[4] https://landscape.cncf.io

Table 1: Different treatments implemented in OXN

Name	Purpose	Tool
Fault Treatment:		
Pause	Simulates an unresponsive service by suspending all processes in a service	DOCKER
Kill	Simulates a service crash	DOCKER
NetworkDelay	Injects network delay on an interface	TC[5]
PacketLoss	Injects packet loss on an interface	TC
PacketCorruption	Injects packet corruption on an interface	TC
Stress	Simulates resource exhaustion by injecting stressors	STRESS-NG[6]
Instrumentation Treatment:		
MetricSamplingRate	Changes the sampling interval for metrics	COLLECTOR
TracingSamplingStrategy	Samples traces based on a given Strategy	COLLECTOR
TracingSamplingRate	Samples traces based on a given sampling rate	COLLECTOR

enable observability experiments. Thus, each can be replaced or extended to adapt to different SUE or treatments. For example, we used DOCKER COMPOSE for the orchestrator. However, we could have also used a KUBERNETES-based orchestrator or one based on CLOUDFORMATION. For the runner, we rely on an PYTHON interface that can be used to manipulate the SUE. For the load generator, we use the LOCUST framework, and for the observers, we implemented interfaces to query JAEGER and PROMETHEUS for now.

Fault and instrumentation *treatments* are defined in a flexible treatment model. Table 1 shows the fault and instrumentation treatments implemented in our prototype. In addition to these treatments, custom treatments can be added easily by implementing a new treatment class against our treatment interface. This allows OXN to be extended with arbitrary fault scenarios or with new instrumentation approaches. Thus, allowing practitioners to build up a large library of relevant faults and treatments to test with OXN.

Response Variables are captured through multiple components. An *observer* component captures experiment information, e.g., execution start and end timestamps. The *responses* component takes in metric and trace data, tracking defined variables and also implements different labeling strategies, depending on the data type. A *store* component writes all captured information as a binary data format that is readable by most data analysis solutions. A *reporter* component can generate a machine-readable experiment report to give engineers a quick overview of the effectiveness of the examined observability design.

To calculate the fault visibility metrics OXN offers an extensible interface, enabling developers to seamlessly integrate their own *fault detection* mechanisms. This could range from threshold-based alerting techniques to more advanced anomaly detection models. By default, we include a logistic regression classifier with OXN, which can be automatically trained to detect faults for a given accuracy threshold, making it readily available for developers to use out-of-the-box. However, developers and observability practitioners can also integrate their own fault detection mechanism, enabling them to test both their observability configuration and their fault detection mechanisms separately. In this way, OXN can also be used to evaluate fault detection mechanisms using real observability data instead of synthetic traces commonly used so far by research.

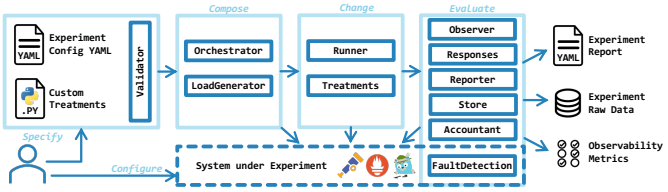

Figure 3: System architecture of OXN

Lastly, the *accountant* component contains functionality to estimate costs of the given observability configuration based on CPU usage. This component could again be extended to include other cost metrics, such as storage or memory overhead.

6 APPLICABILITY AND EXEMPLARY OBSERVABILITY DESIGN ASSESSMENT

Our approach and tooling allows practitioners to evaluate the observability of faults in their microservice application. Besides establishing the effectiveness of the current configuration, they can also use it to reason between observability design alternatives by weighing the observability-cost trade-offs. In this section, we demonstrate the applicability our approach by conducting an exemplary evaluation of the observability of a popular open source microservice application.

6.1 SUE Setup

As our system under experiment (SUE), we use the *OpenTelemetry Astronomy Shop Demo*[7] microservice application, which is a community project intended to illustrate the use of different observability methods and tools in a near real-world environment. The application consists of 20 core application services plus four dedicated for observability services. We chose this application because it covers a wide range of languages and frameworks across the cloud-native application stack. It is instrumented to collect traces across the whole application, leveraging both automatic and custom instrumentation. Besides traces, it is also instrumented to collect several metrics, including base container health metrics and custom metrics added manually by developers. It is a very suitable SUE to showcase OXN, since it offers such a broad spectrum of observability tuning knobs.

For the experiments, we deploy a fork of the application[8] to ensure compatibility with the fault injection functionality of OXN and to enhance container runtime monitoring. We run OXN against the SUE on a cloud-based virtual machine (8vCPUs, 32GB Memory). Figure 4 applies our model for observability design decisions (see 3) and shows the snapshot of our SUE at the time of the baseline measurement. We target our experiments on the recommendation service, which mirrors a real-life scenario for developers who, after creating a new service for their application, are faced with instrumentation and configuration decisions to be able to observe this service effectively.

[5] Linux traffic control
[6] Linux kernel load and stress testing tool

[7] `https://github.com/open-telemetry/opentelemetry-demo`
[8] See the OXN repo for details.

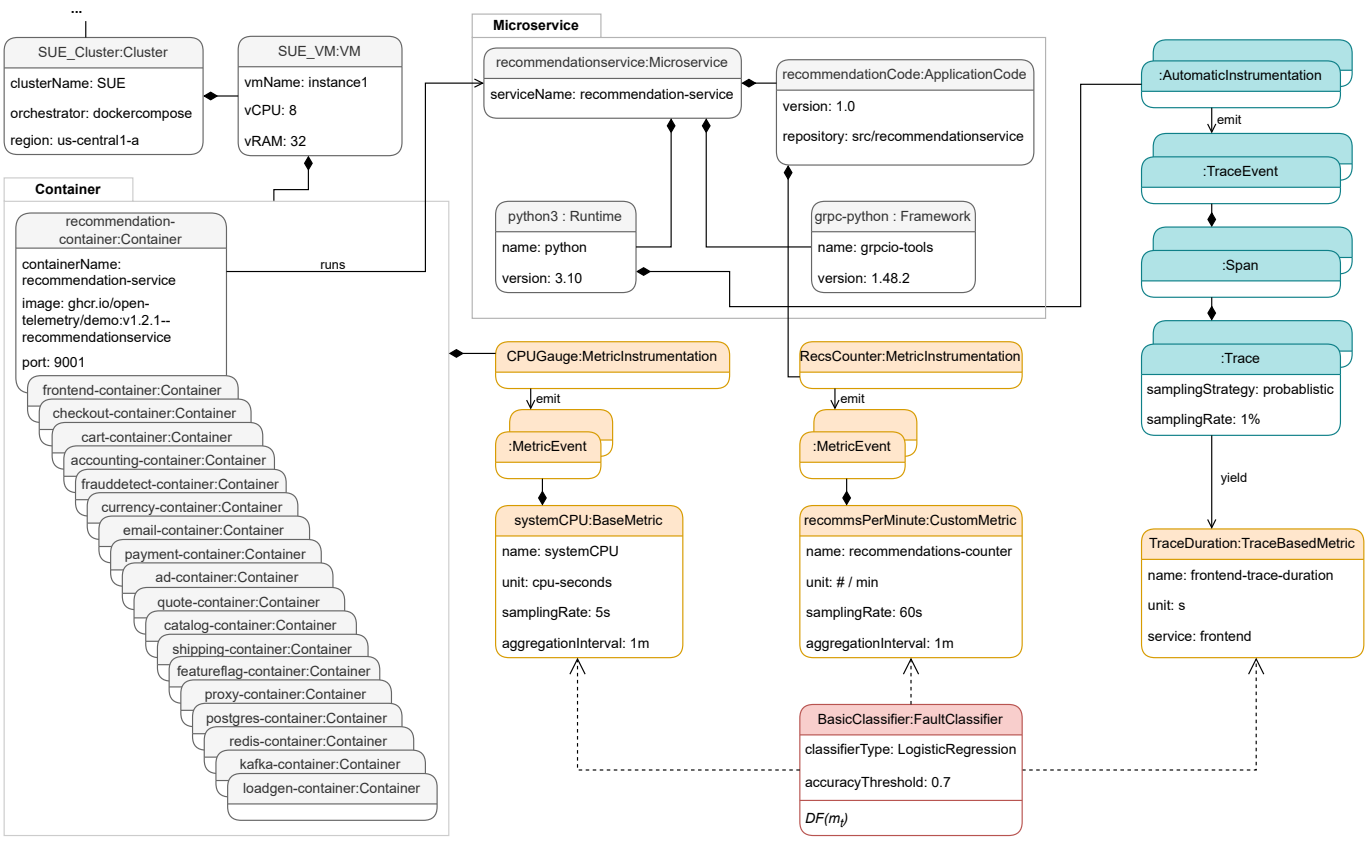

Figure 4: SUE with baseline observability configuration

Our baseline configuration monitors the overall CPU utilization of the system (`systemCPU`) by collecting and aggregating measurements of the `CPUGauges` of every container. By default, base metrics are configured with a 5s sampling rate. Further, the baseline also collects the custom metric `recommsPerMinute`, through an instrumentation point that the developers added to the recommendation application code. This is configured with a 60s sampling rate by default. For tracing, the application is instrumented with both automatic and custom instrumentation, however the recommendation service in question leverages only automatic instrumentation. The tracing sampling strategy is set to a probabilistic sampler[9] with a 1% sampling rate.

As a fault detection mechanism, we use the logistic regression classifier that is shipped with OXN out-of-the-box. Logistic regression is conceptually simple, interpretable, fast to train on large datasets and only requires tuning one hyperparameter. For training, we split the experiment data into training and test sets and ensure an equal class balance of fault and non-fault labels via an oversampling technique. We further preprocess the telemetry data by z-score normalization. After training, we evaluate the classifier's detection function ($DF(m_t)$) on the test data and compute classification accuracy with a threshold of 0.7 as our implementation of α for fault visibility.

[9] `https://opentelemetry.io/docs/specs/otel/trace/tracestate-probability-sampling/`

6.2 *Evaluating the Baseline - Results*

We investigate the observability of our SUE with three different types of faults, `Pause`, `PacketLoss` and `NetworkDelay`. For each fault, we run a 10min experiment under constant load (50 concurrent users) and repeat each experiment 10 times.

Figure 5 plots the metrics we collected over the different experiments with our baseline observability configuration. Table 2 shows the accuracy of the classifier, averaged over the ten experiment runs, and the resulting fault visibility scores. Because we set our classifier threshold α to 0.7, everything that falls below this threshold will not be identified as fault by our fault detection mechanism, and is therefore invisible. As we can see, faults like the pause treatment, which simulates an unresponsive service, manifest themselves quite prominently across metrics. Packet loss is also present in the baseline observability configuration, though with a lower fault coverage (FC) score, since it only manifests in `systemCPU`. Lastly, faults like delay, which could, for example, simulate a faulty cache, are barely visible in the plots and are not detected by our fault detection mechanism, since the detection function doesn't reach the threshold for any of the collected metrics.

6.3 *Evaluating Design Alternatives - Results*

To improve the fault observability of our application, we consider three observability design alternatives (see Figure 6). The first (Fig. 6A) is to increase the sampling rate of the

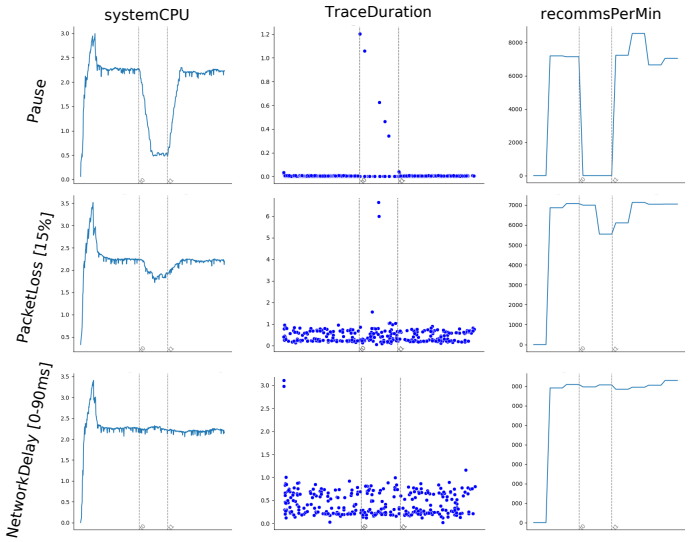

Figure 5: Experiments run against the SUE using OXN, showing how different faults appear visually, similar to how a developer would see them in a dashboard. Note how the Pause fault is visible in all metrics. PacketLoss is noticeable in systemCPU but less pronounced in other metrics, with NetworkDelay not being visible at all. In Figure 7, we see how the changes to the observability configuration proposed in Figure 6 affect these metrics.

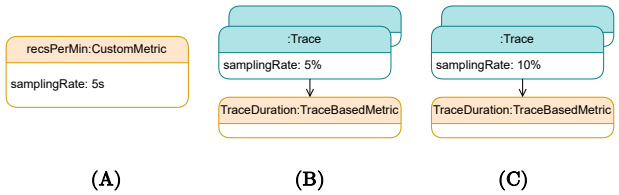

Figure 6: Observability design alternatives under consideration

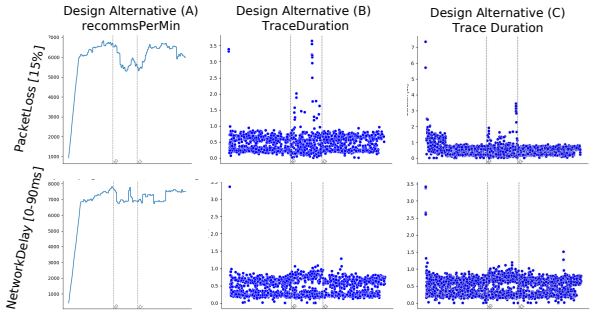

Figure 7: Visual changes to fault visibility across design alternatives. Recognize the now more pronounced effect of PacketLoss on *recommsPerMin*, with NetworkDelay still not affecting this metric significantly. NetworkDelay causes a slight noticeable increase in TraceDuration for the longer Traces for both design alternatives B and C.

Table 2: Fault visibility metrics for default observability configuration

	CPU Util.	Trace Dur.	#Recs/min	FC
Pause	1 (DF = 0.83)	1 (1.00)	1 (0.89)	3/3
PacketLoss [15%]	1 (0.86)	0 (0.61)	0 (0.42)	1/3
NetworkDelay [0-90ms]	0 (0.50)	0 (0.61)	0 (0.43)	0/3
				OFO = 2/3

Table 3: Effects of the Design Alternatives on the fault observability metrics

	PacketLoss [15%]	NetworkDelay [0-90ms]	ΔFC	ΔOFO
(A)	1 (DF=0.72)	0 (0.59)	+1	0
(B)	0 (0.58)	1 (0.77)	+1	+1
(C)	0 (0.60)	1 (0.70)	+1	+1

Table 4: Cost of the Design Alternatives in CPU time [s]

	Baseline SUE	(A)	(B)	(C)
recomm.-service	143.30	150.74	143.86	144.63
otel-col	37.38	37.49	39.05	39.99
prometheus	9.01	9.07	9.07	9.48
jaeger	2.14	2.17	5.70	7.96
total	191.83	199.47	197.68	202.06
overhead	-	+3.98%	+3.05%	+5.33%

application-level recommendation counter, which by default is set to only report once every minute. The other options under consideration are to increase the tracing sampling rate to either 5% (Fig. 6B) or 10% (Fig. 6C).

For each observability design alternative, we repeated the `PacketLoss` and `NetworkDelay` experiments, keeping everything else about the SUE and experiment setup the same. Figure 7 plots the collected metrics after the changes outlined above. Table 3 shows the results regarding fault observability improvements while table 4 reveals the costs associated with carrying out each of these observability design decisions (as CPU time [s] summed across the affected services).

From these results, we can conclude that increasing the interval for the recommendation counter provides better fault coverage for `PacketLoss` faults, an increase from 1/3 to 2/3, but does not affect the OFO, since the `NetworkDelay` fault is still not visible. The second option provides better fault coverage for `NetworkDelay` faults, an increase from 0/3 to 1/3, as well as an increase to the OFO = 3/3, now that the delay fault is finally visible. The third option performs similar to the second one in the fault observability metrics. When looking at observability-related trade-offs in the form of cost (see also table 4), we see that while B are C viable options to improve OFO, B is tied to a lower cost than C, with 3.05% overhead instead of 5.33%.

Thus, by applying the model, metrics and tooling presented in the paper, we can find a design decision with better fault observability, and we can immediately get a first performance impact assessment. A decision maker can now judge if the higher cost is acceptable and implement the change. Otherwise, more observability experiments can be performed to find a better solution. Besides that, the model also provides developers and practitioners with a common language to understand, discuss and document their observability design decisions.

7 LIMITATIONS AND FUTURE WORK

While this paper represents an initial step in the development of observability metrics and the exploration of optimization strategies, it is important to acknowledge certain limitations. First, the approach relies on simulation and isolated experiments, which may not fully capture the complexities and faults faced in real-world applications. For this purpose, we deliberately designed OXN with extensibility in mind, allowing developers to integrate their own load curves and custom faults. In the future, we aim to further validate the approach under real loads and more extensive fault scenarios.

Second, the individual metrics proposed for fault observability were deliberately kept clear and flexible, to enable a technology-independent assessment. A generalization to more complex fault detection systems might not be possible without high coupling to the reporting mechanism. Still, we consider our approach validated as we were able to show the impact of different configuration options on the generated observability data, evident both in the graphical plots and in the classifier score. As part of our ongoing effort to expand observability assessments, we aim to enhance our tool by incorporating support for alerting systems and Mean Time To Detect (MTTD). Besides this, we also aim to improve trade-off analysis by incorporating other cost metrics beyond CPU utilization.

Third, the current implementation of OXN has some limitations, notably the absence of certain features like support for logs[10], KUBERNETES integration, among others. We aim to address these as we continue to utilize our tool in further observability assessments. In future work, we plan to explore how observability decisions can be optimized and automated. Taking our metrics as a foundation, our next objective is to delve into intelligent and learning approaches to optimize and tune observability configuration parameters.

8 CONCLUSION

As observability becomes an increasingly indispensable property of microservice applications, and configuration becomes increasingly complex, there is also a growing concern regarding how to best configure and instrument an application. To enable developers and practitioners to assess the suitability of different designs, we set out to make observability a testable and quantifiable system property, similar to software quality measures like test coverage.

In this paper, we presented an approach for assessing and improving the fault observability of cloud-based microservice applications. Our approach consists of a model for understanding and documenting the observability design space, a set of metrics to assess fault observability, and a tool that provides quantitative evidence for observability design decisions beyond gut-feeling or professional intuition. We showed how this approach can be used in practice, with an application of the model to document the observability design space of a state-of-the-art cloud-based microservice application. Further, we evaluated multiple designs, revealing the until now unexplored observability-cost trade-offs. We aim to improve OXN by adding support for more platforms such as Kubernetes, by integrating the tool in CI pipelines, and by increasing the automation and intelligence of the assessment process. With the help of other practitioners, the model and tooling can also be extended to cover more fault scenarios and more diverse microservice environments. With this work, we lay the foundation toward a systematic method for supporting observability design decisions in cloud-native microservice applications.

[10] At the time of implementation, OPENTELEMETRY had not finalized logs.

ACKNOWLEDGEMENTS

Funded by the European Union (TEADAL, 101070186). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union. Neither the European Union nor the granting authority can be held responsible for them.

REFERENCES

[1] Tarek M. Ahmed, Cor-Paul Bezemer, Tse-Hsun Chen, Ahmed E. Hassan, and Weiyi Shang. Studying the effectiveness of application performance management (apm) tools for detecting performance regressions for web applications: An experience report. In Proc. of the 13th Intl. Conf. on Mining Software Repositories, page 1–12, 2016.

[2] Ali Basiri, Niosha Behnam, Ruud de Rooij, Lorin Hochstein, Luke Kosewski, Justin Reynolds, and Casey Rosenthal. Chaos engineering. IEEE Software, 33(3):35–41, 2016.

[3] David Bermbach, Jörn Kuhlenkamp, Akon Dey, Arunmoezhi Ramachandran, Alan Fekete, and Stefan Tai. Benchfoundry: A benchmarking framework for cloud storage services. In Intl. Conf. on Service-Oriented Computing, pages 314–330, 2017.

[4] David Bermbach, Erik Wittern, and Stefan Tai. Cloud Service Benchmarking: Measuring Quality of Cloud Services from a Client Perspective. Springer, Cham, 2017.

[5] Maria C. Borges, Sebastian Werner, and Ahmet Kilic. Faaster troubleshooting - evaluating distributed tracing approaches for serverless applications. In 2021 IEEE Intl. Conf. on Cloud Engineering, pages 83–90, 2021.

[6] Madalina Dinga, Ivano Malavolta, Luca Giamattei, Antonio Guerriero, and Roberto Pietrantuono. An empirical evaluation of the energy and performance overhead of monitoring tools on docker-based systems. In Intl. Conf. on Service-Oriented Computing, pages 181–196, 2023.

[7] Dominik Ernst and Stefan Tai. Offline trace generation for microservice observability. In IEEE 25th Intl. Enterprise Distributed Object Computing Workshop, pages 308–317, 2021.

[8] Martin Fowler and James Lewis. Microservices, 2014.

[9] Google Architecture Center. Devops measurement: Monitoring and observability, 2023.

[10] Stefan Haselböck and Rainer Weinreich. Decision guidance models for microservice monitoring. In IEEE Intl. Conf. on Software Architecture Workshops, pages 54–61, 2017.

[11] Victor Heorhiadi, Shriram Rajagopalan, Hani Jamjoom, Michael K. Reiter, and Vyas Sekar. Gremlin: Systematic resilience testing of microservices. In 2016 IEEE 36th Intl. Conf. on Distributed Computing Systems, pages 57–66, 2016.

[12] Andrea Janes, Xiaozhou Li, and Valentina Lenarduzzi. Open tracing tools: Overview and critical comparison, 2022.

[13] Jonathan Kaldor, Jonathan Mace, Michał Bejda, Edison Gao, Wiktor Kuropatwa, Joe O'Neill, Kian Win Ong, Bill Schaller, Pingjia Shan, Brendan Viscomi, Vinod Venkataraman, Kaushik Veeraraghavan, and Yee Jiun Song. Canopy: An end-to-end performance tracing and analysis system. In Proc. of the 26th Symp. on Operating Systems Principles, page 34–50, 2017.

[14] John Klein, Ian Gorton, Laila Alhmoud, Joel Gao, Caglayan Gemici, Rajat Kapoor, Prasanth Nair, and Varun Saravagi. Model-driven observability for big data storage. In 2016 13th Working IEEE/IFIP Conference on Software Architecture (WICSA), pages 134–139, 2016.

[15] Jörn Kuhlenkamp and Markus Klems. Costradamus: A cost-tracing system for cloud-based software services. In Intl. Conf. on Service-Oriented Computing, pages 657–672, 2017.

[16] Bowen Li, Xin Peng, Qilin Xiang, Hanzhang Wang, Tao Xie, Jun Sun, and Xuanzhe Liu. Enjoy your observability: an industrial survey of microservice tracing and analysis. Empirical Software Engineering, 27(1):1–28, 2022.

[17] Wubin Li, Yves Lemieux, Jing Gao, Zhuofeng Zhao, and Yanbo Han. Service mesh: Challenges, state of the art, and future research opportunities. In 2019 IEEE Intl. Conf. on Service-Oriented System Engineering, pages 122–1225, 2019.

[18] Christopher S. Meiklejohn, Andrea Estrada, Yiwen Song, Heather Miller, and Rohan Padhye. Service-level fault injection testing. In ACM Symp. on Cloud Computing, page 388–402, 2021.

[19] Sasho Nedelkoski, Jasmin Bogatinovski, Ajay Kumar Mandapati, Soeren Becker, Jorge Cardoso, and Odej Kao. Multi-source distributed system data for ai-powered analytics. In Eur. Conf. on Service-Oriented and Cloud Computing, pages 161–176, 2020.

[20] S. Niedermaier, F. Koetter, A. Freymann, and S. Wagner. On observability and monitoring of distributed systems – an industry interview study. In Intl. Conf. on Service-Oriented Computing, pages 36–52, 2019.

[21] Chadarat Phipathananunth and Panuchart Bunyakiati. Synthetic runtime monitoring of microservices software architecture. In 2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC), volume 02, pages 448–453, 2018.

[22] David Georg Reichelt, Stefan Kühne, and Wilhelm Hasselbring. Overhead comparison of opentelemetry, inspectit and kieker. In Symp. on Software Performance, 2021.

[23] Raja R. Sambasivan, Ilari Shafer, Jonathan Mace, Benjamin H. Sigelman, Rodrigo Fonseca, and Gregory R. Ganger. Principled workflow-centric tracing of distributed systems. In ACM Symp. on Cloud Computing, SoCC '16, page 401–414, 2016.

[24] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Technical report, Google Research, 2010.

[25] Marcio Silva, Michael R. Hines, Diego Gallo, Qi Liu, Kyung Dong Ryu, and Dilma da Silva. Cloudbench: Experiment automation for cloud environments. In Intl. Conf. on Cloud Engineering, pages 302–311, 2013.

[26] Jesper Simonsson, Long Zhang, Brice Morin, Benoit Baudry, and Martin Monperrus. Observability and chaos engineering on system calls for containerized applications in docker. Future Generation Computer Systems, 122:117–129, 2021.

[27] Damian A. Tamburri, Marcello M. Bersani, Raffaela Mirandola, and Giorgio Pea. Devops service observability by-design: Experimenting with model-view-controller. In Eur. Conf. on Service-Oriented and Cloud Computing, pages 49–64, 2018.

[28] Guilherme Vale, Filipe Figueiredo Correia, Eduardo Martins Guerra, Thatiane de Oliveira Rosa, Jonas Fritzsch, and Justus Bogner. Designing microservice systems using patterns: An empirical study on quality trade-offs. In 2022 IEEE 19th International Conference on Software Architecture (ICSA), pages 69–79, 2022.

[29] He Zhang, Shanshan Li, Zijia Jia, Chenxing Zhong, and Cheng Zhang. Microservice architecture in reality: An industrial inquiry. In 2019 IEEE International Conference on Software Architecture (ICSA), pages 51–60, 2019.

[30] Long Zhang, Brice Morin, Benoit Baudry, and Martin Monperrus. Maximizing error injection realism for chaos engineering with system calls. IEEE Transactions on Dependable and Secure Computing, 19(4):2695–2708, 2022.